CO2_Audio

June 28, 2025

Dela Cruz, Cloie Sofia R. Morales, Rayleen Claire G. AM1

1 PART 1: SETTING UP AND LOADING SOUND FILES

Get Tools Ready

[78]:
```
# Librosa is a python package commonly used for music and audio analysis.
# The package provides tools used to analyze and manipulate audio and music.
# In this project, we will use this package for feature extraction, audio␣
 ↪analysis, and music information retrival.
import librosa

# The display module is part of the librosa library.
# It provides functions for visualizing audio data through functions like␣
 ↪waveshow()
# (to visualize waveforms) and specshow() (to create spectograms).
import librosa.display

# This line of code will import the numpy library.
# NumPy is a library in Python for the computation and manipulation of data.
# Using the 'as' syntax, we were able to assign the alias 'np' to numpy to␣
 ↪reference its functions easier throughout the script.
import numpy as np

# This line of code will import the pandas library into the jupyter notebook.
# Pandas is a python package that we will use to analyse and manipulate our␣
 ↪dataset.
# Using the 'as' syntax, we were able to assign the alias 'pd' to panda to␣
 ↪reference its functions easier throughout the script.
import pandas as pd

# Pyplot is a module within the Matplotlib library in Python.
# It provides a MATLAB-like way of generating figures and axes.
# Using the 'as' syntax, we were able to assign the alias 'plt' to matplotlib.
 ↪pyplot to reference its functions easier throughout the script.
import matplotlib.pyplot as plt
```

```
# This line of code will import the StandardScaler class from the Preprocessing␣
 ↪module from the sklearn library.
# This class will be used in part 5 where we will scale the features by␣
 ↪removing the mean and scaling ot unit variance.
from sklearn.preprocessing import StandardScaler

# This line of code imports the soundfile library which is used to read and␣
 ↪write audio files in Python.
# However, since we are using librosa (librosa.load()) in our code, we don't␣
 ↪really need this import directly.
# Librosa already uses soundfile as one of its backend functionality for␣
 ↪loading audio.
import soundfile

# Os is python's built-in module to interact with the operating system
# We will not be using this module within the code because we will not be␣
 ↪working with directories or dyanamic paths.
import os
```

Configure Audio Files

[79]:
```
# This line of code defines "audio_path_jh" as the file name for our first␣
 ↪audio file we will analyze.
# Jumbo Hotdog is a pop song by Masculados.
audio_path_jh = 'jumbohotdog.wav'

# This line of code defines "audio_path_wimy" as the file name for the second␣
 ↪audio file we will analyze.
# When I Met You is an R&B song by Apo Hiking Society.
# We are storing these names as variables so that we may reuse them in our code␣
 ↪when we are loading and processing our audio.
audio_path_wimy = 'whenimetyou.wav'
```

[80]:
```
# The variable label1 stores the genre label of the first audio file.
# Pop indicates the genre of Jumbo Hotdog.
label1 = 'Pop'

# The variable label2 stores the genre label of the second audio file.
# These labels are added to the final table in part 5 to identify which genre␣
 ↪each row (audio) belongs to.
label2 = 'R&B'
```

[81]:
```
# The variable output_csv stores the name of the CSV file where we will save␣
 ↪our extracted and scaled audio features in part 5.
# At the end of the program, we will be able to create a file called␣
 ↪audio_features.csv with all of our processed data.
output_csv = 'audio_features.csv'
```

Load Audio

[82]:
```
# This line of code will display a message in the output to tell us that the␣
 ↪program is starting to load the first audio.
# This will improve the readability of our output.
print("Loading first audio file...")

# This line of code will attempt to use librosa.load() to load the audio data␣
 ↪from the path stored in audio_path_jh.
# The output will be stored in x (a NumPy array with the audio waveform) and␣
 ↪xsr (the sampling rate or samples per second which is usually 22050)
try:
    x, xsr = librosa.load(audio_path_jh)
# This line of code catches exceptions/errors that arise when trying ot load␣
 ↪the first audio file.
# This except block will print an error message saying what went wrong and stop␣
 ↪the program using SystemExit to prevent the program from continuing with bad␣
 ↪data.
except Exception as e:
    print("Error loading first file:", e)
    raise SystemExit
```

```
Loading first audio file...
```

[83]:
```
# This line of code will display a message in the output to tell us that the␣
 ↪program is starting to load the second audio.
# This will improve the readability of our output.
print("Loading second audio file...")

# This line of code will attempt to use librosa.load() to load the audio data␣
 ↪from the path stored in audio_path_wimy.
# The output will be stored in y (a NumPy array with the audio waveform) and␣
 ↪ysr (the sampling rate or samples per second which is usually 22050)
try:
    y, ysr = librosa.load(audio_path_wimy)
# This line of code catches exceptions/errors that arise when trying ot load␣
 ↪the second audio file.
# This except block will print an error message saying what went wrong and stop␣
 ↪the program using SystemExit to prevent the program from continuing with bad␣
 ↪data.
except Exception as e:
    print("Error loading second file:", e)
    raise SystemExit
```

```
Loading second audio file...
```

[84]:
```
# This line of code prints the sample rate of the first audio and the duration␣
 ↪of the audio calculated as len(x)/xsr.
# Since len(x) is the total number of samples, dividing by the sample rates␣
 ↪gives the duration in seconds (formatted to two decimals with 2f).
print(f"First file - Sample rate: {xsr}, Duration: {len(x)/xsr:.2f} seconds")
```

```
First file - Sample rate: 22050, Duration: 44.06 seconds
```

[85]:
```
# This line of code prints the sample rate of the second audio and the duration␣
 ↪of the audio calculated as len(x)/xsr.
# Since len(x) is the total number of samples, dividing by the sample rates␣
 ↪gives the duration in seconds (formatted to two decimals with 2f).
print(f"Second file - Sample rate: {ysr}, Duration: {len(y)/ysr:.2f} seconds")
```

```
Second file - Sample rate: 22050, Duration: 55.14 seconds
```

2 PART 2: VISUALIZING SOUND (WAVEFORMS AND SPECTOGRAMS)

Show Waveforms (Sound Pressure Over Time)

[86]:
```
# This line of code creates a new figure (canvas) for plotting.
# The number 15 idicates the width in units and the number 6 indicates the␣
 ↪height also in units.
plt.figure(figsize=(15, 6))

# This line of code creates the first subplot in a 2-row and 1-column layout.
# This indicates that we will be stacking 2 plots vertically and that the plot␣
 ↪will be in the top slot.
plt.subplot(2, 1, 1)

# This line of code draws the waveform of the first audio.
# The parameter x is the waveform arraw we extracted earlier while sr=xsr is␣
 ↪the sample rate that will in mapping the waveform correctly to time.
# The diagram will show the sound pressure (amplitude) over time.
# Waveshow() is a function from the librosa.display module which helps␣
 ↪visualize the waveform in a time domain.
# It can either display a raw samples-based view or an amplitude-envelope view␣
 ↪of the signal (Librosa Development Team, 2025a).
librosa.display.waveshow(x, sr=xsr)

# This line of code adds a title to the first plot which is "Waveform: Pop."
# This will help in the readability of our figure.
plt.title("Waveform: Pop")

# This line of code will now indicate that we are now working with our second␣
 ↪plot in the 2-row layout.
```

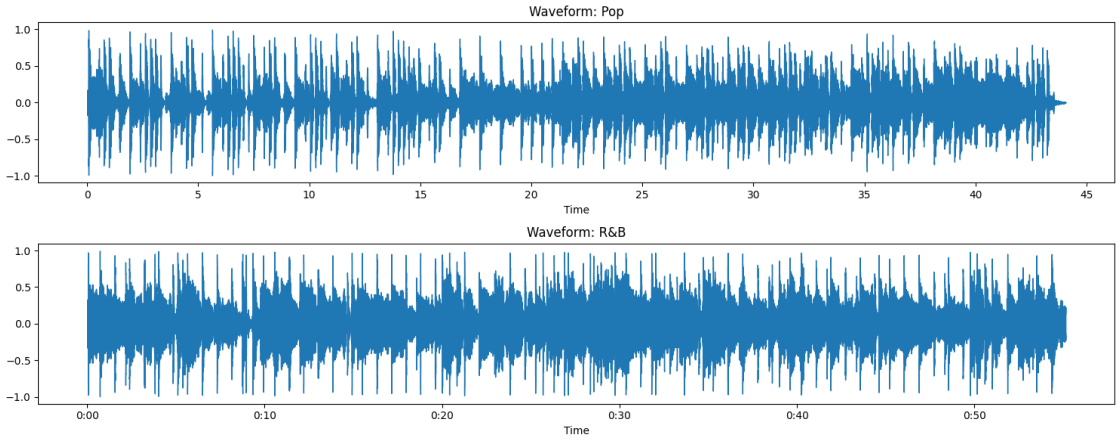
```
# This subplot is where the second waveform will be displayed.
plt.subplot(2, 1, 2)

# This line of code draws the waveform of the second audio.
# y is the waveform of the R&B file while sr=ysr is the sample rate of the second file.
librosa.display.waveshow(y, sr=ysr)

# This line of code adds a title to the second plot which is "Waveform: R&B"
# This will help in the readability of our figure.
plt.title("Waveform: R&B")

# This line of code automatically adjusts the spacing between plots so the titles and axes don't overlap.
# This helps in maintaining the neatness of our figure.
plt.tight_layout()

# This line of code tells the program to display the full figure.
# This figure will of course include the two subplots.
plt.show()
```



Interpretation of Waveforms

In the first waveform, it displays frequent high peaks, sharp veritcal spikes, and very little flat areas. This indicates that the pop song is energetic and using consistent beats/instrumentals. The lack of flat areas tells us that there is very little silence in the song while the sharp spikes suggest percussive elements like drums, claps, or synth hits. These are all characteristics of pop music.

Meanwhile, the second waveform has more variation in amplitude compared to the pop song. There are sections with lower intensity indicated by the spacings in the signal. There are also less sharp spikes and softer transitions between the different segments. These all fit the R&B song because it is smoother, more melodic/dynamic than the pop song. The waveform tells us that the song might include vocals with pauses and softer instruments which are used for emotional build-up in

the song.

Show Spectograms (Frequency Over Time)

[87]:
```
# This line of code creates a new figure (canvas) for plotting.
# The number 12 idicates the width in units and the number 6 indicates the␣
 ↪height also in units.
plt.figure(figsize=(12, 6))

# This line of code creates the first subplot in a 2-row and 1-column layout.
# This indicates that we will be stacking 2 plots vertically and that the plot␣
 ↪will be in the top slot.
plt.subplot(2, 1, 1)

# This line of code performs a Short-Time Fourier Transform (STFT) on the␣
 ↪waveform x.
# According to Xuan (2023), STFT takes small pieces of the given signal and␣
 ↪analyze the Discrete Fourier Transform (DFT) of each segment.
# The DTF takes a discrete signal with data points and computes its frequency␣
 ↪representation.
# This will help us analyze how frequency content evolves over time.
# The variable X stores a matrix of shape (STFT coefficients, frames).
X = librosa.stft(x)

# This line transforms the raw frequency data into a format that would look␣
 ↪better on a graph.
# np.abs(X) get the magnitude of the complex numbers in X. np.abs() computes␣
 ↪the absolute value of each complex number to give us the amplitude of each␣
 ↪frequency at each time step.
# librosa.amplitude_to_db() converts the linear amplitude values into decibel␣
 ↪scale. This makes quiet sounds more visible in the figure.
# Simply put, X_db  will store a 2D array of the spectogram in decibels.
# When plotted, it will show frequency(vertical) vs time(horizontal) vs␣
 ↪loudness in dB (color/intensity) (Librosa Development Team, 2025).
X_db = librosa.amplitude_to_db(np.abs(X))

# This line of code displays the spectogram of the first audio file.
# The x_axis='time' shows time while the y_axis='hz" shows frequency (vertical).
librosa.display.specshow(X_db, sr=xsr, x_axis='time', y_axis='hz')

# This line of code adds a colorbar.
# This colorbar will tell us the amplitude (loudness) of each frequency in␣
 ↪decibels.
plt.colorbar(format="%+2.0f dB")

# This line adds a title to the first subplot.
# This will help in the readability and neatness of the figure.
```

```
plt.title("Spectrogram: Pop")

# This line of code indicates that the program will now work on the second␣
 ↪subplot.
# This subplot will display the spectogram for the R&B song.
plt.subplot(2, 1, 2)

# This line of code will perform STFT on the waveform y.
# Similarly above, the variable Y stores a matrix of shape (STFT coefficients,␣
 ↪frames).
Y = librosa.stft(y)

# This line of code transforms the raw frequency data of Y into a format␣
 ↪suitable for the figure.
# Y_db will store a 2D array of the spectogram in decibels ready for plotting.
Y_db = librosa.amplitude_to_db(np.abs(Y))

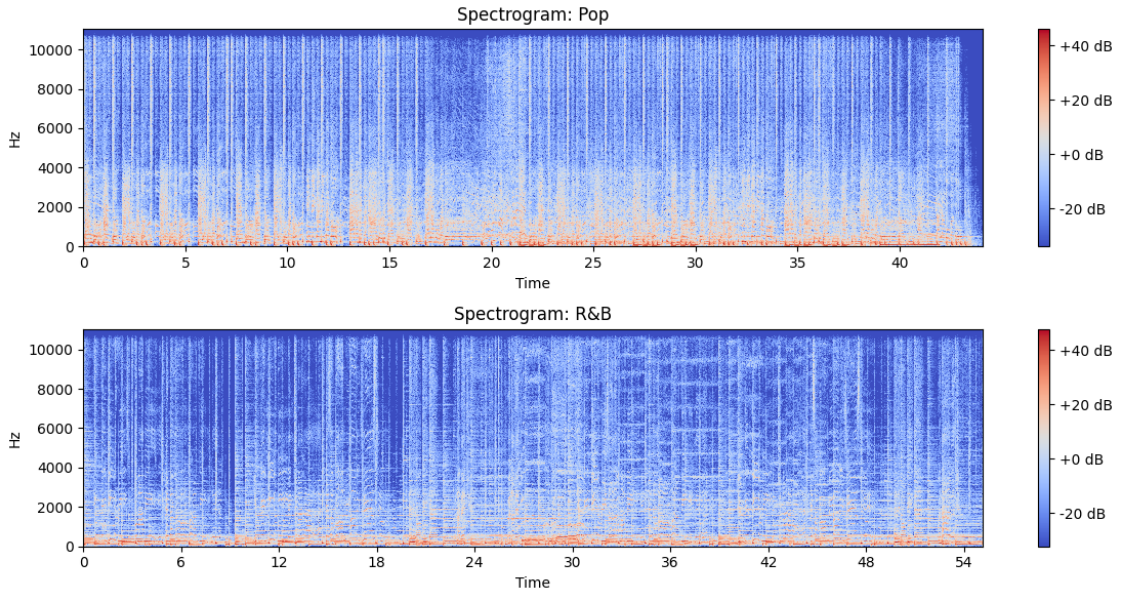
# This line of code displays the spectogram of the second audio file.
# The x_axis='time' shows time while the y_axis='hz" shows frequency (vertical).
librosa.display.specshow(Y_db, sr=ysr, x_axis='time', y_axis='hz')

# This line of code adds a colorbar to the second subplot.
# This colorbar will tell us the amplitude (loudness) of each frequency in␣
 ↪decibels.
plt.colorbar(format="%+2.0f dB")

# This line adds a title to the second subplot.
# This will help in the readability and neatness of the figure.
plt.title("Spectrogram: R&B")

# This line of code automatically adjusts the spacing between plots so the␣
 ↪titles and axes don't overlap.
# This helps in maintaining the neatness of our figure.
plt.tight_layout()

# This line of code tells the program to display the full figure.
# This figure will of course include the two subplots.
plt.show()
```



Interpretation of Spectograms

In the first spectogram, the strong and frequent vertical stripes indicate sharp, repetitive sounds like drums or synths in the song. The red/orange lines near 0-2000 Hz suggest that bass and kick drums are frequently used. Similarly with its waveform, the first audio file's spectograms show that energy is very dense and continuous, with very little space between the different sound events.

On the other hand, the spectogram for the R&B song has distinct sections with varying loudness and frequency. At the early part of the song, lower frequency indicates vocals, basslines, or smooth instruments. The vertical patterns are also more spaced out, suggesting softer beats.

3 PART 3: EXTRACTING KEY SOUND FEATURES

Make "Feature Extractor" Helper

[88]:
```
# Through this line of code, we are defining a function named␣
 ↪"Extract_Sound_Feature."
# This function will take in three arguments: the raw waveform, the smaples per␣
 ↪second, and a boolean value to control whether the function will return␣
 ↪averaged features or time-varying features.
def Extract_Sound_Features(sound_data, sample_rate, return_mean=True):

    # This line of code initializes an empty list called features.
    # This is where we will store the averaged values.
    if return_mean:
        features = []

        # Zero Crossing Rate
```

```
        # The Zero Crossing Rate (ZCR) is useful for detecting percussive or␣
↪noisy sound.
        # It measures how often a signal's value changes (positive to negative␣
↪and vice versa) within a given point in time.
        # The librosa.feature.zero_crossing_rate() function computes the ZCR of␣
↪an audio time series (Librosa Development Team, 2025b).
        zcr = librosa.feature.zero_crossing_rate(sound_data)
        # np.mean() function averages the ZCR over time.
        # This value will then be appended to the features list.
        features.append(np.mean(zcr))

        # Spectral Centroid
        # The spectral centroid indicates where the center of mass of the␣
↪spectrum is located.
        # This is commonly associated as a measure of the brighness of the␣
↪sound (OpenAE, 2025).
        # This means that the higher the vvalues, the brighter the sound.
        centroid = librosa.feature.spectral_centroid(y=sound_data,␣
↪sr=sample_rate)
        # np.mean() takes the mean across time to summarize centroid.
        # This values will then be appended to our features list.
        features.append(np.mean(centroid))

        # Spectral Rolloff
        # The Spectral Rolloff is defined as the "frequency below which a␣
↪specified percentage (85%) of the total spectral energy lies" (Music␣
↪Information Retrieval, 2025a).
        # This is useful for distinguishing voiced vs unvoiced sounds.
        rolloff = librosa.feature.spectral_rolloff(y=sound_data, sr=sample_rate)
        # Again, np.mean() takes the average over time.
        # This value will be appended into our features list.
        features.append(np.mean(rolloff))

        # MFCCs
        # The Mel Frequency Cepstral Coefficients (MFCCSs) of a signal are "a␣
↪small set of features which concisely describe the over shape of a spectral␣
↪envelope."
        # These are useful to describe timbre (Music Information Retrieval,␣
↪2025).
        # This line of code tells the program to extract 13 MFCCs.
        mfcc = librosa.feature.mfcc(y=sound_data, sr=sample_rate, n_mfcc=13)
        # Because mfcc is a matrix with shape (13, time_steps), this line of␣
↪code tells the prgram to loop through each coefficient and average it over␣
↪time.
        # These will be added to our features list.
        for coeff in mfcc:
```

```
            features.append(np.mean(coeff))

        # Chroma Frequencies
        # According to Lyons (2020), chroma splits frequencies into 12 pitch␣
↪classes. This line of code meaningfully categorizes the different pitches in␣
↪the song into these categories.
        # This helps in identifying musical harmony in the song.
        chroma = librosa.feature.chroma_stft(y=sound_data, sr=sample_rate)
        # This like of code averages each pitch class intensity.
        # After this, these averages are added into the features list.
        for chroma_band in chroma:
            features.append(np.mean(chroma_band))

        # This line of code returns a list of 28 features.
        # It contains 1 ZCR, 1 Centroid, 1 Rolloff, 13 MFCCs, and 12 Chroma␣
↪values.
        return features

    # When return_mean is equal to false, the function will return time-varying␣
↪features.
    # This line of code initializes a dictionary called features to store␣
↪feature vectors over time).
    else:
        features = {}

        # This line of code gets the full ZCR vector across time frames.
        # This stores the feature as a sequence over time.
        features['zcr'] = librosa.feature.zero_crossing_rate(sound_data)[0]

        # This line of code gets the full centroid values over time.
        # This line of code also stores the value into the features dictionary.
        features['centroid'] = librosa.feature.spectral_centroid(y=sound_data,␣
↪sr=sample_rate)[0]

        # This line of code gets the full rolloff curve of the audio.
        # It also stores it in the features dictionary.
        features['rolloff'] = librosa.feature.spectral_rolloff(y=sound_data,␣
↪sr=sample_rate)[0]

        # This line of code only retrieves that first MFCC coefficient over␣
↪time.
        # It also stores it in the features dictionary.
        features['mfcc1'] = librosa.feature.mfcc(y=sound_data, sr=sample_rate,␣
↪n_mfcc=13)[0]
```

```
        # This line of code get only the first chroma bin over time which is␣
 ↪the C pitch class.
        # It also stores it in the features dictionary.
        features['chroma1'] = librosa.feature.chroma_stft(y=sound_data,␣
 ↪sr=sample_rate)[0]

        # This line of code returns the time-varying dictionary.
        # This will allow us to create a plot a displays the different features␣
 ↪of an audio over time.
        return features
```

Extract Features of Audio Files

[89]:
```
# This line of code prints a message to let us know that the program is now␣
 ↪computing summary statistics for the audio features.
# This helps in the readability of our program.
print("Extracting averaged features...")
```

```
Extracting averaged features…
```

[90]:
```
# This line of code calls our custom function on the first audio and tells it␣
 ↪to average each feature into a single value.
# The output features1 is a list of 28 scalar features of the first audio file.
features1 = Extract_Sound_Features(x, xsr, return_mean=True)

# This line of code prints out the list of computed values for the first audio␣
 ↪file.
# This is helpful in verifying that the features are extracted correctly.
print("Features for first file:", features1)
```

```
Features for first file: [np.float64(0.06431291779175448),
np.float64(1781.0286389372889), np.float64(3759.6768442521734),
np.float32(-125.38319), np.float32(115.75021), np.float32(-5.463808),
np.float32(14.69019), np.float32(10.540217), np.float32(4.228912),
np.float32(9.723722), np.float32(0.3506707), np.float32(-9.452802),
np.float32(-3.0103037), np.float32(-5.7159104), np.float32(2.98727),
np.float32(3.2876976), np.float32(0.5480726), np.float32(0.40267813),
np.float32(0.3727785), np.float32(0.4299242), np.float32(0.4698794),
np.float32(0.4136657), np.float32(0.36609772), np.float32(0.3246914),
np.float32(0.3570601), np.float32(0.38610652), np.float32(0.43220532),
np.float32(0.58881474)]
```

[91]:
```
# This line of code calls our custom function on the second audio and tells it␣
 ↪to average each feature into a single value.
# The output features1 is a list of 28 scalar features of the second audio file.
features2 = Extract_Sound_Features(y, ysr, return_mean=True)
```

```
# This line of code prints out the list of computed values for the second audio␣
 ↪file.
# This is helpful in verifying that the features are extracted correctly.
print("Features for second file:", features2)
```

```
Features for second file: [np.float64(0.040945106907894734),
np.float64(1336.347718049304), np.float64(2770.797378700658),
np.float32(-154.22937), np.float32(122.11575), np.float32(9.199652),
np.float32(37.847485), np.float32(29.608904), np.float32(14.047023),
np.float32(-0.17960182), np.float32(-1.7715473), np.float32(-5.2614384),
np.float32(-13.094833), np.float32(-11.798), np.float32(-6.386773),
np.float32(-6.2516127), np.float32(0.36224192), np.float32(0.31966504),
np.float32(0.33676276), np.float32(0.26613632), np.float32(0.24489957),
np.float32(0.34971604), np.float32(0.2942355), np.float32(0.31083912),
np.float32(0.23364654), np.float32(0.31082115), np.float32(0.483247),
np.float32(0.3480529)]
```

[92]:
```
# This line of code prints a message that will indicate that the next part of␣
 ↪the code will now extract time-variant features.
# This will help in the readability of our code.
print("Extracting time variant features...")
```

```
Extracting time variant features…
```

[93]:
```
# This line of code calls our custom function and tells it to extract the␣
 ↪time-variant features of the first audio file.
# This returns a dictionary features_time1 with values that are arrays over␣
 ↪time.
features_time1 = Extract_Sound_Features(x, xsr, return_mean=False)

# This line of code will print out the values of features_time1.
# This is useful for debugging and plotting.
print("Time variant features for first file:", features_time1)
```

```
Time variant features for first file: {'zcr': array([0.09423828, 0.09960938,
0.10351562, …, 0.02734375, 0.02099609,
       0.01318359], shape=(1898,)), 'centroid': array([2258.9637869 ,
3162.80563709, 2201.8446604 , …,  840.91193661,
        786.18415168,  896.33776591], shape=(1898,)), 'rolloff':
array([5092.60253906, 7611.98730469, 6373.828125  , …, 1873.38867188,
       1539.62402344, 1668.82324219], shape=(1898,)), 'mfcc1': array([-185.71043
, -106.74404 ,  -47.273167, …, -444.48557 ,
       -458.79187 , -474.44748 ], shape=(1898,), dtype=float32), 'chroma1':
array([0.81137013, 0.85032225, 0.77657497, …, 0.6033726 , 0.67523724,
       0.5778983 ], shape=(1898,), dtype=float32)}
```

[94]:
```
# This line of code calls our custom function and tells it to extract the␣
 ↪time-variant features of the second audio file.
```

```
# This returns a dictionary features_time2 with values that are arrays over␣
 ↪time.
features_time2 = Extract_Sound_Features(y, ysr, return_mean=False)

# This line of code will print out the values of features_time2.
# This is useful for debugging and plotting.
print("Time variant features for second file:", features_time2)
```

```
Time variant features for second file: {'zcr': array([0.01171875, 0.01953125,
0.03173828, …, 0.02392578, 0.02636719,
       0.02099609], shape=(2375,)), 'centroid': array([ 726.56817796,
1210.57723742, 1288.37208877, …, 1040.78144524,
       1069.70501833, 1384.25653621], shape=(2375,)), 'rolloff':
array([1055.12695312, 2465.55175781, 2993.11523438, …, 2045.65429688,
       2088.72070312, 2723.95019531], shape=(2375,)), 'mfcc1':
array([-222.31314, -113.40899,  -68.1368 , …, -241.98657, -227.56065,
       -190.79733], shape=(2375,), dtype=float32), 'chroma1': array([1.        ,
1.        , 0.9500108 , …, 0.08881508, 0.08472048,
       0.107706  ], shape=(2375,), dtype=float32)}
```

4 PART 4: VISUALIZING INDIVIDUAL FEATURES

[95]:
```
# This part of the code aims to generate the time axes for our subplots.
# This line of code takes the total number of frames of the␣
 ↪features_time1['zcr'].
# The range() function creates a sequence of number from 0 to N-1 for each␣
 ↪frame.
# frames1 will now contain a sequence of numbers for our axes.
frames1 = range(len(features_time1['zcr']))

# This line of code functions that same as the one above but for the second␣
 ↪audio file.
# This gives us frames2 which is the timeline in terms of frame numbers for the␣
 ↪second audio file.
frames2 = range(len(features_time2['zcr']))

# This line of code converts frames1 which are indices into actual time values␣
 ↪in seconds.
# sr=xsr provides the sample rate for accurate time conversions.
time1 = librosa.frames_to_time(frames1, sr=xsr)

# This line of code converts frames2 which are indices into actual time values␣
 ↪in seconds.
# sr=xsr provides the sample rate for accurate time conversions.
# Now, we can align time with features for plotting.
time2 = librosa.frames_to_time(frames2, sr=ysr)
```

```
# This line of code create a new figure window for plotting.
# It indicates a width of 14 units and a height of 20 units.
# We will add five vertically stacked plots in this figure.
plt.figure(figsize=(14, 20))

# 1. Zero Crossing Rate

# This line of code divides the figure into a grid of five rows by one column.
# It also activates the first subplot (the top one).
plt.subplot(5, 1, 1)

# This line of code plots a line graph of the zero-crossing rate of the first␣
 ↪audio file over time.
# time1 is the x-axis in seconds while features_time1['zcr'] is the y-axis.
plt.plot(time1, features_time1['zcr'], label='Pop')

# This line of code is the same as the one above but for the second audio file.
# Now two lines will appear in the same subplot.
plt.plot(time2, features_time2['zcr'], label='R&B')

# This line of code will add a title to the plot.
# This will help in the readability of the output.
plt.title("Zero Crossing Rate")

# This labels the x-axis.
# The label will be "Time (s)" indicating that the time will be in seconds.
plt.xlabel("Time (s)")

# This line of code labels the y-axis as "ZCR."
# This stands for Zero Crossing Rate.
plt.ylabel("ZCR")

# This line of code tells the program to display a legend in the subplot.
# This will help in distinguishing between the Pop and R&B lines.
plt.legend()

# This line of code adds a grid behind the plot.
# This will help visually track values.
plt.grid(True)

# 2. Spectral Centroid

# This line of code tells the program to work on another subplot.
# This will help us modify and create the second subplot of the 5-row layout.
plt.subplot(5, 1, 2)
```

```
# This line of code plots a line graph of the spectral centroid of the first␣
 ↪audio file over time.
# time1 is the x-axis in seconds while features_time1['centroid'] is the y-axis.
plt.plot(time1, features_time1['centroid'], label='Pop')

# This line of code plots a line graph of the spectral centroid of the second␣
 ↪audio file over time.
# time2 is the x-axis in seconds while features_time2['centroid'] is the y-axis.
plt.plot(time2, features_time2['centroid'], label='R&B')

# This line of code will add a title to the plot.
# This will help in the readability of the output.
plt.title("Spectral Centroid")

# This labels the x-axis.
# The label will be "Time (s)" indicating that the time will be in seconds.
plt.xlabel("Time (s)")

# This line of code labels the y-axis as "Hz."
# This stands for Hertz.
plt.ylabel("Hz")

# This line of code tells the program to display a legend in the subplot.
# This will help in distinguishing between the Pop and R&B lines.
plt.legend()

# This line of code adds a grid behind the plot.
# This will help visually track values.
plt.grid(True)

# 3. Spectral Rolloff

# This line of code tells the program to work on another subplot.
# This will help us modify and create the third subplot of the 5-row layout.
plt.subplot(5, 1, 3)

# This line of code plots a line graph of the spectral rolloff of the first␣
 ↪audio file over time.
# time1 is the x-axis in seconds while features_time1['rolloff'] is the y-axis.
plt.plot(time1, features_time1['rolloff'], label='Pop')

# This line of code plots a line graph of the spectral rolloff of the second␣
 ↪audio file over time.
# time2 is the x-axis in seconds while features_time2['rolloff'] is the y-axis.
plt.plot(time2, features_time2['rolloff'], label='R&B')

# This line of code will add a title to the plot.
```

```
# This will help in the readability of the output.
plt.title("Spectral Rolloff")

# This labels the x-axis.
# The label will be "Time (s)" indicating that the time will be in seconds.
plt.xlabel("Time (s)")

# This line of code labels the y-axis as "Hz."
# This stands for Hertz.
plt.ylabel("Hz")

# This line of code tells the program to display a legend in the subplot.
# This will help in distinguishing between the Pop and R&B lines.
plt.legend()

# This line of code adds a grid behind the plot.
# This will help visually track values.
plt.grid(True)

# 4. MFCC Coefficient 1

# This line of code tells the program to work on another subplot.
# This will help us modify and create the fourth subplot of the 5-row layout.
plt.subplot(5, 1, 4)

# This line of code plots a line graph of the MFCC Coefficient 1 of the first␣
 ↪audio file over time.
# time1 is the x-axis in seconds while features_time1['mfcc1'] is the y-axis.
plt.plot(time1, features_time1['mfcc1'], label='Pop')

# This line of code plots a line graph of the MFCC Coefficient 1 of the second␣
 ↪audio file over time.
# time2 is the x-axis in seconds while features_time2['mfcc1'] is the y-axis.
plt.plot(time2, features_time2['mfcc1'], label='R&B')

# This line of code will add a title to the plot.
# This will help in the readability of the output.
plt.title("MFCC Coefficient 1")

# This labels the x-axis.
# The label will be "Time (s)" indicating that the time will be in seconds.
plt.xlabel("Time (s)")

# This line of code labels the y-axis as "Amplitude."
# This will help in the readability of our code.
plt.ylabel("Amplitude")
```

```
# This line of code tells the program to display a legend in the subplot.
# This will help in distinguishing between the Pop and R&B lines.
plt.legend()

# This line of code adds a grid behind the plot.
# This will help visually track values.
plt.grid(True)

# 5. Chroma Band 1

# This line of code tells the program to work on another subplot.
# This will help us modify and create the fifth subplot of the 5-row layout.
plt.subplot(5, 1, 5)

# This line of code plots a line graph of the Chroma Band 1 of the first audio␣
 ↪file over time.
# time1 is the x-axis in seconds while features_time1['chroma1'] is the y-axis.
plt.plot(time1, features_time1['chroma1'], label='Pop')

# This line of code plots a line graph of the Chroma Band 1 of the second audio␣
 ↪file over time.
# time2 is the x-axis in seconds while features_time2['chroma1'] is the y-axis.
plt.plot(time2, features_time2['chroma1'], label='R&B')

# This line of code will add a title to the plot.
# This will help in the readability of the output.
plt.title("Chroma Band 1")

# This labels the x-axis.
# The label will be "Time (s)" indicating that the time will be in seconds.
plt.xlabel("Time (s)")

# This line of code labels the y-axis as "Intensity."
# This will help in the readability of our code.
plt.ylabel("Intensity")

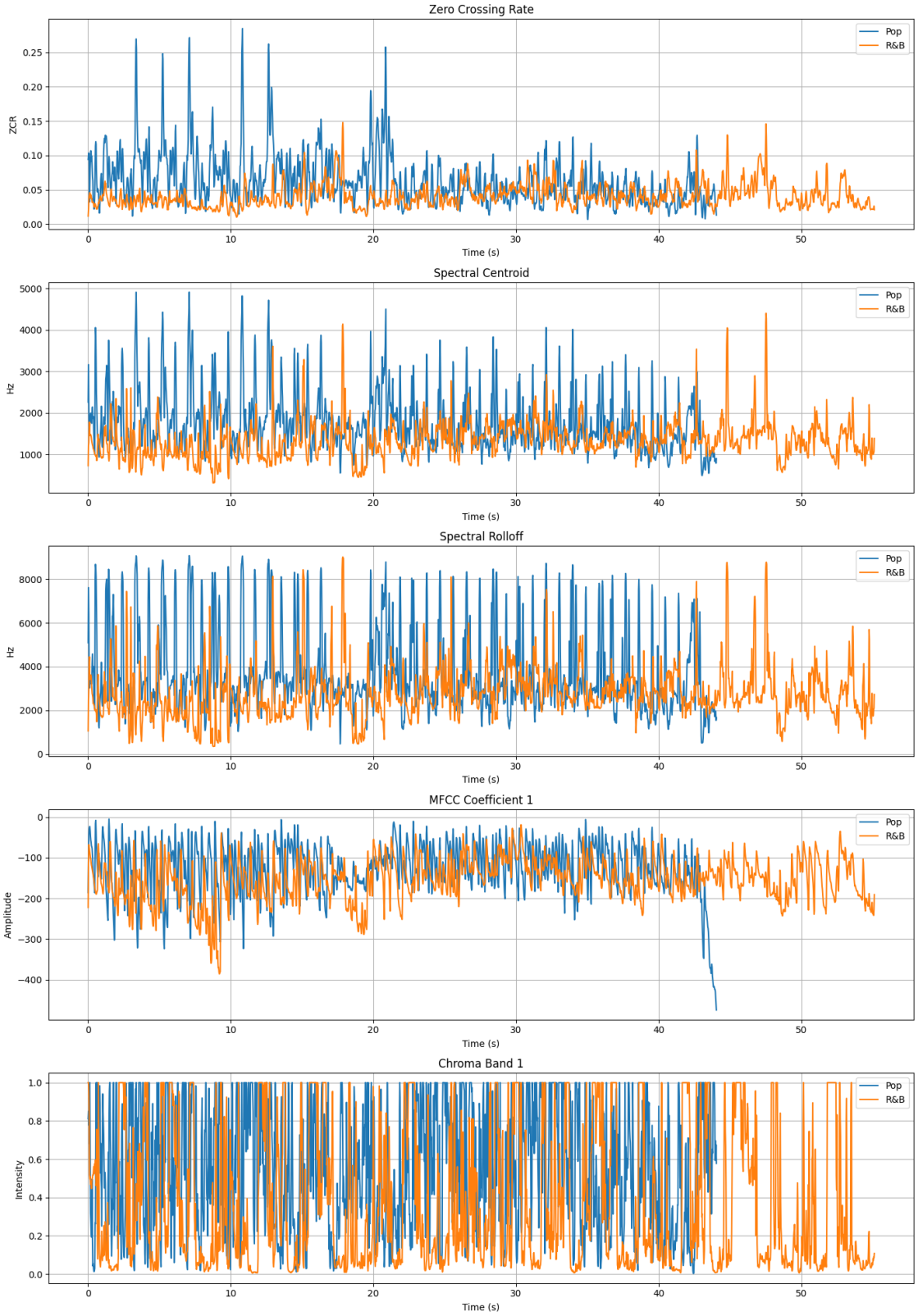
# This line of code tells the program to display a legend in the subplot.
# This will help in distinguishing between the Pop and R&B lines.
plt.legend()

# This line of code adds a grid behind the plot.
# This will help visually track values.
plt.grid(True)

# This line of code automatically adjusts the spacing between plots so the␣
 ↪titles and axes don't overlap.
# This helps in maintaining the neatness of our figure.
```

```
plt.tight_layout()

# This line of code tells the program to display the full figure.
# This figure will of course include the five subplots.
plt.show()
```


Zero Crossing Rate
ZCR
Time (s)
Pop
R&B
Spectral Centroid
Hz
Time (s)
Pop
R&B
Spectral Rolloff
Hz
Time (s)
Pop
R&B
MFCC Coefficient 1
Amplitude
Time (s)
Pop
R&B
Chroma Band 1
Intensity
Time (s)
Pop
R&B

Interpretation of Different Plots

Zero Crossing Rate: High ZCR often indicates for noisy or percussive sounds. The Pop track shows higher and more frequent ZCR spikes, suggesting shaper or more percussive elements in the audio. On the other hand, the R&B track shows lower ZCR, implying a smoother, more continuous signal with fewer abrupt changes.

Spectral Centroid: Higher spectral centroid values suggest brighter (more high-frequency) sounds. The Pop trakc shows frequent peaks reaching higher frequencies, meaning it likely contains bright instruments or synths. Meanwhile, the R&B track stays lower on average which aligns with the smoother and bass-dominant audio.

Spectral Rolloff: The spectral rolloff reflects the brightness and high-frequency energy of an audio file. Again, the Pop track shows higher and sharper rolloff values, supporting its energetic and brights sounds. The R&B tracks has more stable and lower rolloff curve which fits its more laid-back tone with fewer high frequencies dominating.

MFCC 1: The first MFCC often relates to overall spectral slope. Both genres show a lot of fluctuations, but the R&B track tends to dip lower than the Pop trakc. This could mean deeper tonal characteristics or vocal elements. The Pop line shows more consistency in higher amplitude, possibly due to its consistent and predictable rhythm.

Chroma Band 1: Chroma band 1 corresponds to the first pitch class C. The Pop track displays frequent, strong shifts between high and low values, showing dynamic harmonic movement or chord changes. Meanwhile, the R&B tracks shows slightly more stable and clustered usage.

5 PART 5: ORGANIZING AND SAVING FEATURES

Prepare Data Table (DataFrame)

[96]:
```
# This line prints a message indicating that we are about to start preparing␣
 ↪the feature table.
# This can help us track progress while running the notebook.
print("\nPreparing feature table...")
```

```
Preparing feature table…
```

[97]:
```
# pd.DataFrame() is used to create a DataFrame.
# Wrapping features1 in brackets means that we will create a DataFrame with one␣
 ↪row only.
df1 = pd.DataFrame([features1])

# This line of code adds a new columns called "Genre" to the DataFrame df1.
# The value of label1 which is "Pop" is added to this column.
df1['Genre'] = label1
```

[98]:
```
# This line of code is the same as the one above but for the second audio␣
 ↪file's features.
# This creates a anther single-row DataFrame from the second track.
```

```
df2 = pd.DataFrame([features2])

# This line of code adds a new columns called "Genre" to the DataFrame df2.
# The value of label2 which is "R&B" is added to this column.
df2['Genre'] = label2
```

[99]:
```
# pd.concat() combines the two DataFrames vertically (row-wise).
# ignore_index=True tells the program that the index will be reset to 0 and 1␣
 ↪instead of keeping the original indices.
# This line of code will result in a single table with two rows, onw for each␣
 ↪audio file, and their extracted features + genre.
combined_df = pd.concat([df1, df2], ignore_index=True)
```

[100]:
```
# Again, this line of code prints out a message to separate the output visually.
# This will help in the readability of our code.
print("\nRaw Feature Table:")

# This line of code prints out the entire combined_df DataFrame.
# This is useful for reviewing and checking our dataset.
print(combined_df)
```

```
Raw Feature Table:
          0            1            2           3           4         5  \
0  0.064313  1781.028639  3759.676844 -125.383186  115.750214 -5.463808
1  0.040945  1336.347718  2770.797379 -154.229370  122.115753  9.199652

           6          7          8         9  …        19        20  \
0  14.690190  10.540217   4.228912  9.723722  …  0.429924  0.469879
1  37.847485  29.608904  14.047023 -0.179602  …  0.266136  0.244900

         21        22        23        24        25        26        27  Genre
0  0.413666  0.366098  0.324691  0.357060  0.386107  0.432205  0.588815    Pop
1  0.349716  0.294235  0.310839  0.233647  0.310821  0.483247  0.348053    R&B

[2 rows x 29 columns]
```

Scale Features

[101]:
```
# This line of code prints a message indicating that we will now start to scale␣
 ↪our features.
# This message is helpful for tracking the stages of our code in our notebook.
print("\nScaling features...")
```

```
Scaling features…
```

[102]:
```
# This line of code assigns the function StandardScaler() to the variable␣
 ↪scaler.
# This functions standardizes features by removing the mean and scaling to unit␣
 ↪variance, ensuring all numeric features are on a similar scale.
scaler = StandardScaler()

# combined_df.drop('Genre', axis=1) removes the Genre column temporarily,␣
 ↪because it's categorical (text) and not meant to be scaled.
# fit() calculates the mean and standard deviation of each feature column while␣
 ↪transform() applies the standardization formula to each value.
# scaled_values is no a NumPy array of the same shape as the original feature␣
 ↪table but standardized.
scaled_values = scaler.fit_transform(combined_df.drop('Genre', axis=1))

# This line of code converts the scaled NumPy array back into a Pandas␣
 ↪DataFrame for easier manipulation.
# It also dynamically creates generic column names and avoids using the␣
 ↪original feature names (which were dropped in the scaling process).
scaled_df = pd.DataFrame(scaled_values, columns=[f'Feature{i+1}' for i in␣
 ↪range(scaled_values.shape[1])])

# This line of code reattaches the Genre columns from the original combined_df.
# This allows us to maintain the labels in our scaled dataset.
scaled_df['Genre'] = combined_df['Genre']
```

[103]:
```
# This line of code prints a message to prepare for viewing of the final scaled␣
 ↪table.
# The '\n\ adds spacing for better formatting.
print("\nScaled Feature Table:")
```

```
Scaled Feature Table:
```

[104]:
```
# This line of code displays the complete scaled DataFrame.
# This is the final version of our dataset and it is ready for saving.
print(scaled_df)
```

```
   Feature1  Feature2  Feature3  Feature4  Feature5  Feature6  Feature7  \
0       1.0       1.0       1.0       1.0      -1.0      -1.0      -1.0
1      -1.0      -1.0      -1.0      -1.0       1.0       1.0       1.0

   Feature8  Feature9  Feature10  …  Feature20  Feature21  Feature22  \
0      -1.0      -1.0        1.0  …        1.0        1.0        1.0
1       1.0       1.0       -1.0  …       -1.0       -1.0       -1.0

   Feature23  Feature24  Feature25  Feature26  Feature27  Feature28  Genre
0        1.0        1.0        1.0        1.0       -1.0        1.0    Pop
1       -1.0       -1.0       -1.0       -1.0        1.0       -1.0    R&B
```

```
[2 rows x 29 columns]
```

Save to File

[105]:
```
# This line of code prints a message indicating we are now saving the file.
# This lets us know that the program is starting the save-to-file step.
print("\nSaving to file...")
```

```
Saving to file…
```

[106]:
```
# This line of code begins a try block.
# This is used to catch errors in the saving process.
try:
    # The to_csv() is a Pandas method that writes the DataFrame to a CSV file.
    # The output_csv contains the file name we defined earlier and index=False␣
 ↪tells Pandas to not include the index columns in the file.
    scaled_df.to_csv(output_csv, index=False)

    # This line of code prints the message that confirms to_csv() sucessfully␣
 ↪create the CSV file.
    # The f" string allows the insertion of the variable output_csv directly in␣
 ↪the message.
    print(f"Success! Features saved to: {output_csv}")

# This line of code catches any kind of error that may occur in the try block.
# If an error occurs, this line will print the error message.
except Exception as e: print("Error saving file:", e)
```

```
Success! Features saved to: audio_features.csv
```

[107]:
```
# This line of code prints a final message to indicate that the program has␣
 ↪finished.
# This is helpful to see that the whole process including saving completed␣
 ↪without crashing.
print("DONE!")
```

```
DONE!
```

REFERENCES

- Librosa Development Team. (2025). librosa.amplitude_to_db — librosa 0.11.0 documentation. Librosa.
- Librosa Development Team. (2025a). librosa.display.waveshow . librosa.
- Librosa Development Team. (2025b). librosa.feature.zero_crossing_rate. Librosa.
- Lyons, M. (2020, September 8). Music Information Retrieval. Medium.
- Music Information Retrieval. (2025). mfcc. Music Information Retrieval.
- Music Information Retrieval. (2025a). spectral_features. Music Information Retrieval.
- OpenAE. (2025). Spectral Centroid. OpenAE.

- Xuan, Z. (2023, April 24). Exploring the Short-Time Fourier Transform: Analyzing Time-Varying Audio Signals. Medium.